

Object Detection on Pascal VOC using Faster R-CNN with Soft-NMS

INM705 Deep Learning for Image Analysis

Ibrahim Qubaisi

April 2026

1 Introduction

This project addresses the task of object detection — simultaneously localising and classifying objects within images. Specifically, I detect four classes (person, car, bus, bicycle) using the Pascal VOC 2007 and 2012 datasets [1]. The model family used is CNN/ConvNet-based (Option A), and the problem type is object detection and localisation.

Object detection has evolved from sliding window approaches through the R-CNN family [2], which introduced region-based detection using CNNs. Fast R-CNN [3] improved efficiency by sharing convolutional features across proposals, and Faster R-CNN [4] replaced selective search with a learned Region Proposal Network (RPN), enabling end-to-end training. My implementation builds on Faster R-CNN with a ResNet-101 backbone [5] and Feature Pyramid Network (FPN) [6], which constructs multi-scale feature maps to detect objects at varying sizes.

A known limitation of Faster R-CNN is its reliance on hard Non-Maximum Suppression (NMS), which removes overlapping detections using a fixed IoU threshold. Bodla et al. [7] proposed Soft-NMS, which decays detection scores based on overlap rather than removing them entirely, addressing over-suppression in crowded scenes. I implement and evaluate Soft-NMS as a post-processing improvement.

The base model architecture uses torchvision's Faster R-CNN implementation with pre-trained ImageNet weights. My contributions include: implementing Soft-NMS post-processing to reduce duplicate detections, investigating the effect of NMS threshold on detection quality, applying a StepLR learning rate scheduler to prevent overfitting and improve box precision, comparing training on VOC 2007 versus VOC 2012 to measure the impact of dataset size, evaluating data augmentation strategies and their effect on generalisation, and analysing confidence thresholds for partially visible objects in crowded scenes.

GitHub: https://github.com/IQubaisi/VOC_Detection

Dataset: Pascal VOC 2007 and 2012 from <http://host.robots.ox.ac.uk/pascal/VOC/>

Dataset location: data/VOCdevkit/VOC2007/ and data/VOCdevkit/VOC2012/

2 Methodology

2.1 Model Architecture

I use Faster R-CNN [4] with a ResNet-101 [5] backbone and Feature Pyramid Network (FPN) [6], loaded from torchvision with ImageNet pretrained weights. The architecture consists of four stages (Figure 1): the backbone extracts multi-scale feature maps, FPN combines low-resolution semantically strong features with high-resolution spatially precise features through lateral connections. The Region Proposal Network slides over these feature maps generating candidate proposals with objectness scores using anchors at multiple scales and aspect ratios. RoI Align extracts fixed-size feature representations from each proposal, and the detection head outputs 5 class scores (background + 4 classes) and 4 bounding box coordinates per detection. I chose ResNet-101 over ResNet-50 for deeper feature representations, justified by the limited training data (1,387 filtered images for VOC 2007). Transfer learning from ImageNet was essential — training from scratch with this data size would be insufficient [2].

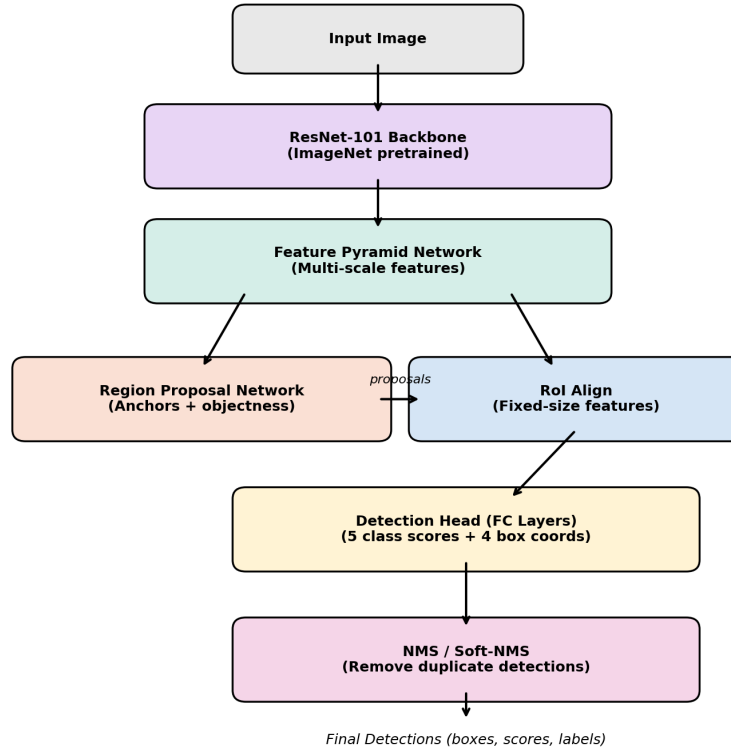


Figure 1: Faster R-CNN architecture with ResNet-101 backbone, FPN, and NMS/Soft-NMS post-processing.

2.2 Training Configuration

I use SGD with momentum=0.9 and weight_decay=0.0005, consistent with the original Faster R-CNN training procedure [4]. Batch size is 2 due to GPU memory constraints with the ResNet-101 backbone. All experiments use `torch.manual_seed(42)` for reproducibility. Checkpoints are saved after every epoch, with the best selected by mAP on the validation set. All experiments are logged to Weights & Biases.

2.3 Loss Function

Faster R-CNN uses a multi-task loss combining classification and bounding box regression:

$$L = L_{cls} + L_{box} \quad (1)$$

where L_{cls} is cross-entropy loss over object classes and L_{box} is Smooth L1 loss:

$$\text{smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \quad (2)$$

Smooth L1 is less sensitive to outliers than L2 loss, making box regression more stable.

2.4 My Contributions

Soft-NMS: Standard NMS removes any detection overlapping a higher-scoring box above a fixed IoU threshold. I implemented Gaussian Soft-NMS [7], which decays scores using:

$$s_i = s_i \times e^{-\text{IoU}(b_i, b_m)^2 / \sigma} \quad (3)$$

where s_i is the detection score, b_m is the highest-scoring box, and σ controls decay strength. This preserves valid nearby detections in crowded scenes.

Learning Rate Scheduling: I applied StepLR, reducing the learning rate by $\gamma = 0.1$ every 5 epochs. This allows large optimisation steps early and progressively finer steps as the model converges, preventing overfitting observed in fixed learning rate experiments.

Data Augmentation: Using the albumentations library [8], I applied random horizontal flips ($p = 0.5$) and colour jitter (brightness, contrast, saturation $\pm 20\%$, $p = 0.5$) during training only. An initial experiment with aggressive RandomResizedCrop was found to hurt performance and was subsequently removed.

Dataset Comparison: I trained identical configurations on Pascal VOC 2007 (1,387 filtered images) and VOC 2012 (2,666 filtered images) to isolate the effect of dataset size on detection performance.

3 Results

All models are evaluated on Pascal VOC 2007 validation set (1,324 images) using Mean Average Precision (mAP) at IoU thresholds of 0.50, 0.75, and 0.50:0.95, computed via torchmetrics with the pycocotools backend.

Table 1: VOC 2007 Results

Experiment	mAP@0.50	mAP@0.75	mAP@0.50:0.95
Baseline ($lr=0.001$)	0.6390	0.2110	0.2946
Hard NMS (thresh=0.3)	0.5653	—	—
Soft-NMS ($\sigma=0.5$)	0.6587	0.2128	0.2995
Scheduler	0.6410	0.2450	0.3038
Scheduler + Soft-NMS	0.6609	0.2455	0.3079
Augmentation	0.6353	0.2406	0.3000
Augmentation + Soft-NMS	0.6520	0.2412	0.3035
Gentle Augmentation	0.6525	0.2524	0.3099
Gentle Augmentation + Soft-NMS	0.6765	0.2559	0.3158

Table 2: VOC 2012 Results

Experiment	mAP@0.50	mAP@0.75	mAP@0.50:0.95
Baseline ($lr=0.001$)	0.7267	0.3436	0.3735
Soft-NMS ($\sigma=0.5$)	0.7333	0.3412	0.3728
Scheduler	0.7357	0.3765	0.3880
Scheduler + Soft-NMS	0.7397	0.3772	0.3883
Aggressive Augmentation	0.7224	0.3253	0.3698
Aggressive Augmentation + Soft-NMS	0.7305	0.3258	0.3709
Gentle Augmentation	0.7418	0.3846	0.3953
Gentle Augmentation + Soft-NMS	0.7459	0.3813	0.3939

NMS Analysis (Table 1, rows 1–3): Reducing the hard NMS threshold from 0.5 to 0.3 dropped mAP by 0.074, demonstrating over-suppression of valid nearby detections. Soft-NMS addressed this by decaying scores rather than removing boxes, improving mAP by 0.020 over the baseline without the over-suppression problem.

Scheduler Impact (Tables 1–2): The StepLR scheduler improved mAP@0.75 substantially — from 0.2110 to 0.2450 on VOC 2007 and from 0.3436 to 0.3765 on VOC 2012. This indicates the model produces tighter, more precisely localised boxes when the learning rate is progressively reduced.

Dataset Size (Tables 1–2): VOC 2012 consistently outperformed VOC 2007 across all experiments, with the baseline improving from 0.6390 to 0.7267 mAP@0.50. Notably, Soft-NMS had a smaller effect on VOC 2012 (+0.007) than VOC 2007 (+0.020), suggesting that a model trained on more data produces fewer duplicate detections.

Augmentation Analysis: An initial augmentation experiment using aggressive Random-ResizedCrop reduced mAP on both datasets, suggesting scale distortion is counterproductive for object detection. However, replacing the crop with gentle augmentation (horizontal flip and colour jitter only) produced the best overall result of 0.7459 mAP@0.50 on VOC 2012, ...demonstrating that augmentation strategy — not augmentation itself — is the critical factor. Figure 2 breaks down per-class performance for the best model, revealing car detection achieves the highest AP while bicycle scores lowest due to its smaller size and greater pose

variation.

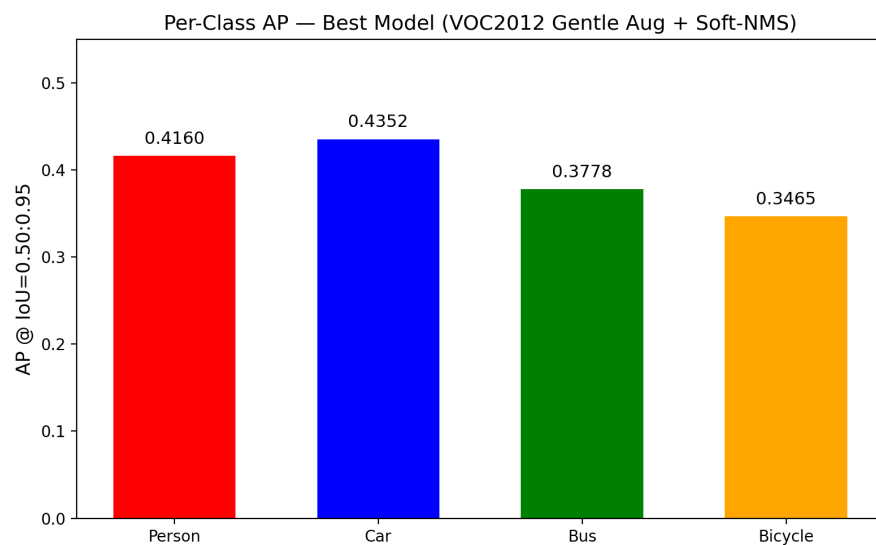
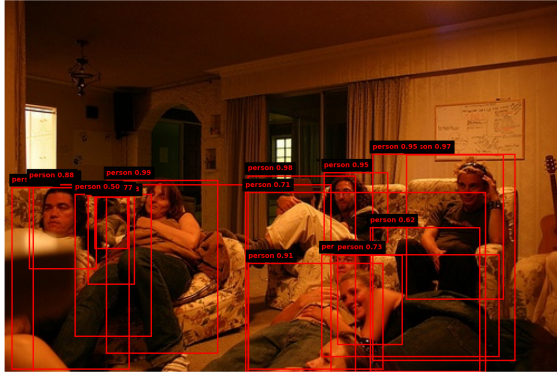
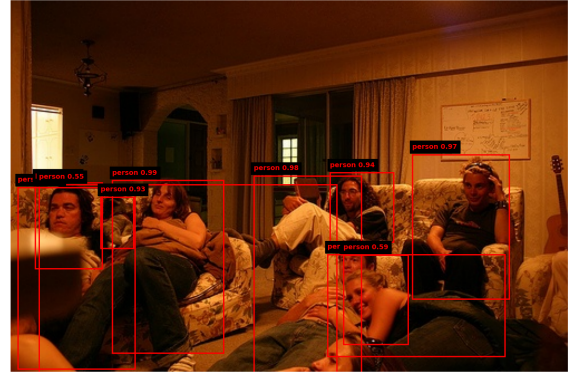


Figure 2: Per-class AP for the best model (VOC 2012, gentle augmentation + Soft-NMS). Car achieves the highest AP (0.4352), while bicycle scores lowest (0.3465), likely due to its smaller size and greater pose variation.

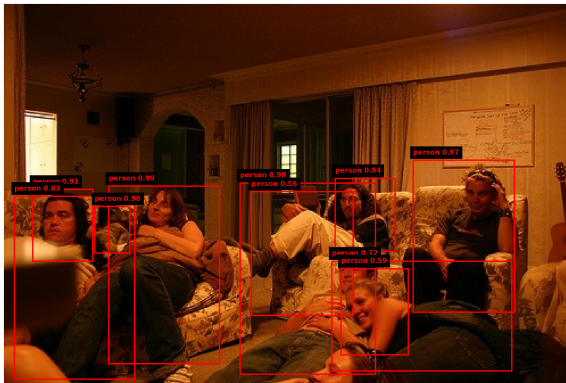
Figure 3 shows the visual progression across experiments on the same crowded scene. Each step progressively reduces duplicate detections, increases confidence scores, and improves localisation precision. A separate confidence threshold experiment (0.3, 0.5, 0.7) revealed that partially visible people are detected at scores of 0.37–0.44 but filtered at the standard 0.5 threshold, suggesting that a dedicated head detector could improve recall for occluded individuals.



VOC2012 baseline — 16 boxes



+ Soft-NMS — 10 boxes

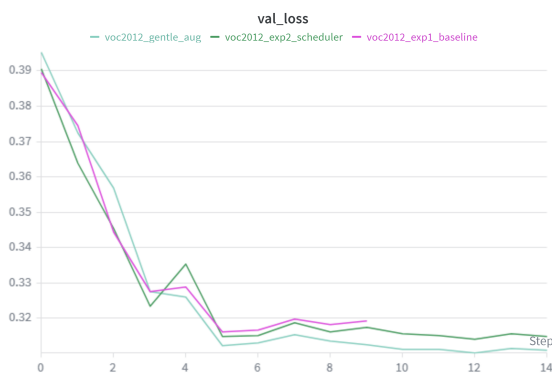


+ Scheduler + Soft-NMS — 10 boxes



+ Gentle Aug + Soft-NMS — 9 boxes best result

Figure 3: Gradual improvement across experiments on VOC 2012, same crowded scene (confidence threshold 0.5). (a) Baseline produces duplicate boxes with mixed confidence. (b) Soft-NMS removes duplicates while preserving distinct detections. (c) Scheduler improves confidence scores (0.91–0.99). (d) Gentle augmentation achieves highest confidence (0.99) and detects partially visible people at 0.73.



Validation loss



Training loss

Figure 4: Training curves from WandB across three VOC 2012 experiments. The baseline (pink) shows the fastest initial drop but plateaus early. The scheduler (green) stabilises after epoch 5 when the learning rate drops, achieving lower final loss. Gentle augmentation (purple) follows a similar trajectory but maintains slightly higher training loss — indicating better generalisation rather than memorisation.

4 Conclusions

This project implemented Faster R-CNN with a ResNet-101 backbone and FPN for object detection on Pascal VOC, detecting person, car, bus, and bicycle across a series of controlled experiments.

The best overall result was achieved using VOC 2012 with StepLR scheduling, gentle data augmentation, and Soft-NMS post-processing, reaching 0.7459 mAP@0.50 — an improvement of 0.1069 over the initial VOC 2007 baseline of 0.6390. Each modification contributed differently to this improvement.

Dataset size had the largest single impact, with VOC 2012 improving over VOC 2007 by approximately 0.09 mAP@0.50 across all configurations, confirming that increasing training data is more effective than architectural or post-processing modifications.

The StepLR scheduler primarily improved box precision — mAP@0.75 improved by 0.034 on VOC 2007 and 0.033 on VOC 2012, indicating tighter bounding box localisation when learning rate decay prevents late-training overshoot.

Soft-NMS consistently improved mAP across all experiments, validating that score decay is superior to binary suppression for handling overlapping detections. Its diminishing effect on VOC 2012 compared to VOC 2007 demonstrates that better-trained models inherently produce fewer duplicate detections.

Aggressive data augmentation with RandomResizedCrop hurt mAP, but gentle augmentation with horizontal flips and colour jitter produced the best overall result, demonstrating that augmentation strategy requires careful tuning to the specific task — removing the destructive crop while keeping beneficial transforms was the key insight.

5 Reflections

Challenges: The most significant challenge was checkpoint management. After migrating from a local directory to a GitHub repository, I lost all original experiment checkpoints as they were excluded by `.gitignore`. Retraining Experiment 2 produced different mAP results (0.5745 vs 0.6410 at the same val loss) before I identified that val loss and mAP do not always correlate — evaluating multiple checkpoints revealed epoch 7 matched the original result at 0.6390. This taught me to always evaluate checkpoints by the task metric (mAP), not training loss alone.

When implementing data augmentation, an initial implementation rebuilt the augmentations pipeline object for every image during loading, causing training to stall for over 40 minutes without completing a single epoch. I identified the bottleneck by profiling single image loads, confirmed the dataset itself loaded in 0.4 seconds, and resolved it by initialising the pipeline once during dataset construction and reusing it for all subsequent image loads — reducing per-image overhead to near zero.

Running experiments on Google Colab introduced further challenges — session timeouts wiped locally stored data, and subprocess-based training caused silent bottlenecks. These were resolved by saving checkpoints directly to Google Drive and using direct shell commands instead of subprocess wrappers. Setting `torch.manual_seed(42)` was added after observing training variance, a step I should have implemented from the start.

Deviations from plan: I initially expected data augmentation to improve performance. Aggressive augmentation (RandomResizedCrop) hurt mAP, leading me to believe augmentation was counterproductive. However, removing the crop and keeping only horizontal flip and colour jitter ultimately produced the best result of the entire project — demonstrating the importance of iterating on negative results rather than abandoning the approach entirely.

Insights: Through the confidence threshold experiment, I observed that partially visible people are detected by the model but at lower confidence scores (0.3–0.6 rather than 0.9+). The model's limitation is not detection capability but detection confidence for occluded objects. A dedicated head detector trained specifically on head annotations would likely address this, as heads remain visible even when bodies are heavily occluded — this is a well-established approach in crowd counting and surveillance applications.

What I would do differently: I would set a random seed and implement checkpoint management from the beginning of the project rather than retroactively. I would also evaluate on VOC 2012 validation when training on VOC 2012 rather than cross-evaluating on VOC 2007 val, which I only recognised as an inconsistency late in the project. Finally, I would experiment with more targeted augmentation strategies such as Mosaic augmentation, which composites four training images into one and naturally creates more partially visible objects without the scale distortion caused by random cropping.

References

- [1] Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2010). The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2), pp.303–338.
- [2] Girshick, R., Donahue, J., Darrell, T. and Malik, J. (2014). Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. *CVPR 2014*.
- [3] Girshick, R. (2015). Fast R-CNN. *ICCV 2015*.
- [4] Ren, S., He, K., Girshick, R. and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS 2015*.
- [5] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
- [6] Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B. and Belongie, S. (2017). Feature Pyramid Networks for Object Detection. *CVPR 2017*.
- [7] Bodla, N., Singh, B., Chellappa, R. and Davis, L.S. (2017). Soft-NMS — Improving Object Detection with One Line of Code. *ICCV 2017*.
- [8] Buslaev, A., Iglovikov, V.I., Khvedchenya, E., Parinov, A., Druzhinin, M. and Kalinin, A.A. (2020). Albumentations: Fast and Flexible Image Augmentations. *Information*, 11(2), p.125.